

IMPLEMENTAÇÃO DE UM PONTO ELETRÔNICO COM O THINGSPEAK UTILIZANDO A PLACA BITDOGLAB

Arthur Lucas Costa Ciriaco da Silva
Área temática: Indústria

RESUMO

O projeto é de suma importância na indústria, pois busca praticidade na hora de gerenciar, tendo em vista que o objetivo central do projeto visa a administração de dados de entrada e saída de funcionários dentro de uma empresa com um sistema de baixo custo. A aplicação conta com um sistema explicativo que realiza envio de dados por meio de protocolos de comunicação e usando como hardware uma placa BitDogLab. Durante o decorrer do desenvolvimento o protótipo conseguiu validar os objetivos iniciais de envio da quantidade de entradas/saídas de múltiplos funcionários.

- Praticidade.
- Gerenciamento.
- Sistema Explicativo.
- Eficiência.
- Baixo custo.

INTRODUÇÃO

O projeto surgiu da necessidade de gerenciar os horários e a quantidade de entradas e saídas dos funcionários dentro de uma empresa. Além disso, muitas soluções de ponto eletrônico disponíveis no mercado possuem custos elevados, tornando-se inacessíveis para pequenos e médios empresários.

Diante desse cenário, este projeto busca oferecer um sistema de baixo custo e eficiente, utilizando materiais acessíveis e de fácil implementação. Além de registrar as entradas e saídas de cada funcionário, o sistema garante um serviço

completo e seguro, atendendo às demandas de empresas que precisam de uma alternativa econômica e funcional para o controle de ponto.

Fig. 1 – Imagem representativa de um chip.

METODOLOGIA

No projeto, foi utilizada uma placa BitDogLab, equipada com um Raspberry Pi Pico W. Os seguintes periféricos foram utilizados: botões, display OLED, Joystick e o módulo de conexão Wi-Fi. Como protocolo de comunicação foi utilizado o HTTP.

Os periféricos de joystick, display oled e botões foram usados para a escolha de opções, onde o funcionário navega por meio de um joystick em uma interface de menu apresentada pelo display OLED, podendo avançar ou retroceder utilizando os botões.

O projeto passou por diversas partes, entre elas: desenvolvimento de um código que conseguisse implementar esses componentes e suportasse o protocolo HTTP, especificamente para tipo de requisições GET, que são utilizadas no envio de dados para o ThingSpeak, também houve uma etapa de testes e aprimoramento.

A coleta de dados é feita manualmente por meio do funcionário que opta por marcar a hora/data da entrada/saída e ao se identificar é mandado uma requisição para o servidor (ThingSpeak), atualizando-o.

RESULTADOS ALCANÇADOS E DISCUSSÕES

Primeiramente, devemos analisar os arquivos presentes no .zip do projeto, que incluem: cliente.c, cliente.h CmakeLists.txt, lwipopts.h, mbedtls_config.h, ssd1306_font.h, ssd1306_i2c.c, ssd1306_i2c.h, ssd1306.h, além do arquivo principal, pico.c que contém a implementação principal do projeto.

Sobre os arquivos:

Lwipopts.h e mbedtls_config.h:

O arquivo lwipopts.h é essencial para configuração do lwIP (Lightweight IP) que é responsável pela comunicação via IP. No projeto é utilizado quando optamos pelo protocolo de comunicação HTTP.

Já o arquivo `mbedtls_config.h` é responsável pela configuração da biblioteca de criptografia leve, garantindo a segurança e o funcionamento para a comunicação.

`client.h` e `client.c`:

O arquivo `client.h` possui as funções essenciais para facilitar a comunicação, enquanto o `client.c` implementa essas funções na prática, detalhando a lógica de conexão, funções de callback e controle de timeouts. A separação entre esses dois arquivos é devido a facilidade de manutenção e entendimento, vale ressaltar que os dois arquivos são essenciais na configuração das funções usadas no código principal (`pico.c`).

Os arquivos que iniciam com `ssd1306`:

São arquivos necessários para o uso do display OLED presente na placa BitDogLab, configuram para o uso do display no código principal (`pico.c`), esses arquivos foram retirados do GitHub da BitDogLab Disponível em: https://github.com/BitDogLab/BitDogLab-C/tree/main/display_oled/inc. Acesso em: 16 de fev. de 2025.

Em uma segunda discussão vamos analisar o diagrama de entidades presentes no sistema embarcado e como foi implementado no projeto:

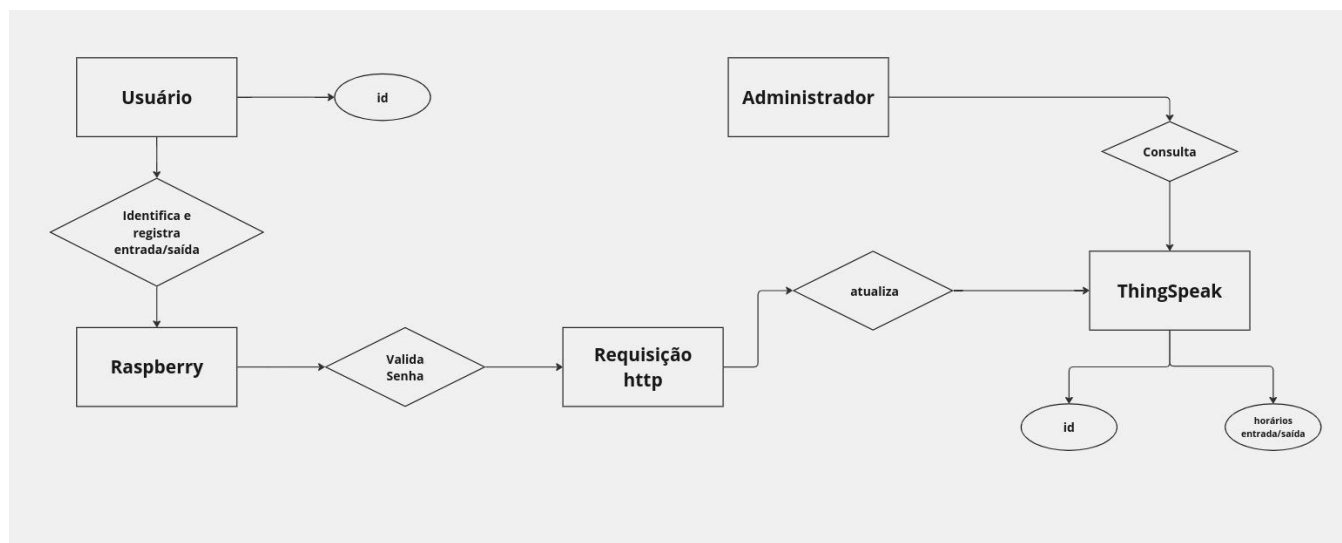


Fig. 1 – Diagrama de Entidades.

Inicialmente, o usuário pode realizar dois tipos de requisições do tipo GET: uma referente a entrada e outra para saída. Após selecionar a opção desejada, ele passa por um processo de identificação. Na primeira versão do protótipo, é possível cadastrar até quatro usuários, que são identificados por um ID. Uma vez cadastrado, os dados de entrada/saída são enviados através da função requisição, que recebe como parâmetros o campo a ser atualizado e o respectivo valor.

Em seguida, essas informações são armazenadas no servidor do Thingspeak, após o envio o usuário pode voltar a tela inicial onde poderá repetir todo o processo. Os dados enviados podem ser analisados facilmente por um administrador que por sua vez pode verificar a data e o horário da entrada ou saída e, posteriormente, validar os dados conforme o ID identificado, vendo o que é condizente ou não.

Após a explicação do funcionamento do sistema, é de suma importância o conhecimento das principais funções de client.c que são essenciais para o uso do ThingSpeak:

Função `connected_wifi`:

É uma função simples que realiza a conexão do Raspberry com o WI-FI por meio do SSID da rede e a senha, ela retorna o status da tentativa de conexão podendo variar entre conectado ou não.

Função cliente_write:

É a responsável por enviar os dados através de uma conexão TCP usando um Protocol Control Block (PCB). Ela garante que os dados sejam escritos e enviados corretamente para o servidor, em seus parâmetros verifica-se o pcb e data (mensagem a ser enviada) são válidos, se algum valor for inválido é retornado falso, dentro da função há o tcp_write que é o responsável por enfileirar os dados para envio, e a função tcp_output que força o envio dos dados enfileirados, se houver erro retorna false, há também um laço de repetição do tipo while que espera a confirmação por meio de sent_callback, e se ultrapassar TCP_WRITE_TIMEOUT_MS um timeout é registrado e a função retorna false, se o envio for bem-sucedido, é exibido uma mensagem de cliente_write concluído.

Falando um pouco mais sobre funções principais do pico.c:

Função requisição:

```
void requisicao(const int field, int valor){
    ip_addr_t server_ip;
    resolve_name("api.thingspeak.com", &server_ip);
    printf("IP resolvido: %s\n", ipaddr_ntoa(&server_ip));
    struct tcp_pcb *pcb = tcp_new();
    client_create(pcb, &server_ip, 80);
    char request[256];
    snprintf(request, sizeof(request),
        "GET /update.json?api_key=QMOVRT1DELYENMZ5&field%d=%d HTTP/1.1\r\n"
        "Host: api.thingspeak.com\r\n"
        "Connection: close\r\n\r\n",
        field, valor);
    client_write(pcb, request);
}
```

Fig. 2 – Função requisição.

A função requisição é a função responsável pela formatação de uma string e responsável pelo envio para a api key "QMOVRT1DELYENMZ5" de acordo com a field e o valor da entrada, no código foi aplicada da seguinte forma:

```
if(estado_A == true){  
    entrada_1 += 1;  
    envio(field_1, entrada_1);  
}
```

Fig. 3 – Parte do código em que é usada a função de envio.

Ao funcionário passar pela etapa da identificação e escolher enviar um dado apertando o botão A acrescentado um na variável de entrada ou de saída, e é enviado o para a field_x a quantidade de entrada/saída_x, assim atualizando o site do ThingSpeak.

Obs.: todas as variáveis de entrada/saída são inicializadas com 0.

Também é de suma importância falar sobre a parte que permite a utilização de um menu:

```
adc_select_input(0);  
adc_y_raw = adc_read();  
if (adc_y_raw > 3000 && last_adc_y_raw <= 3000) {  
    opcao_selecionada = (opcao_selecionada - 1 + OPCOES) % OPCOES;  
}  
if (adc_y_raw < 1000 && last_adc_y_raw >= 1000) {  
    opcao_selecionada = (opcao_selecionada + 1) % OPCOES;  
}  
  
last_adc_y_raw = adc_y_raw;
```

Fig. 4 – Parte do código que permite criação de um menu.

Essa parte do código é essencial na criação de um menu, inicia fazendo o mapeamento do eixo y, logo após faz uma detecção por meio de comparações se o joystick está transitando para cima ou para baixo, se ele ultrapassa o limite é guardado em uma variável que nesse caso é opcao_selecionada que vai receber o valor de uma expressão podendo aumentar ou diminuir, de acordo com a inclinação do joystick, e a parte de % OPCOES vem para trazer à tona um comportamento cíclico.

Obs.: OPCOES é de acordo com a quantidade de opções do menu em questão, no projeto há duas etapas com esse sistema de menu, na escolha de entrada/saída e na escolha do id.

Sobre o arquivo de configuração CmakeLists.txt:

Dentro do arquivo CmakeLists.txt é necessário listar dois principais pontos, sendo eles: o executável (add_executable) e as bibliotecas usadas.

Sobre o executável: eles são requisitados para a execução do sistema, os arquivos necessários para as funções do display (ssd1306_i2c.c), o arquivo necessário para uso das funções de conexão (client.c) e o arquivo pico.c que contém o loop principal.

ficando assim no final:

```
add_executable(pico pico.c ssd1306_i2c.c client.c)
```

Fig. 5 – Parte do CMakeLists.txt referente a adição de executáveis.

Em relação às bibliotecas é de suma importância adicionar as bibliotecas essenciais para o uso do display, do joystick, do temporizador de protocolos de comunicação no caso na figura a seguir mostra sobre o target_link_libraries, onde as 3 primeiras são as bibliotecas usadas basicamente para a conexão, comunicação e depois vem as bibliotecas de hardware.

Ficando assim no final:

```
target_link_libraries(pico
    pico_cyw43_arch_lwip_threadsafe_background
    pico_lwip_mbedtls
    pico_mbedtls
    hardware_adc
    hardware_i2c
    hardware_clocks
    hardware_pwm
)
```

Fig. 6 – Parte do CMakeLists.txt referente a vinculação de bibliotecas.

CONCLUSÃO

O ponto eletrônico usando o ThingSpeak e a placa BitDogLab se mostrou eficiente na utilização, os objetivos foram alcançados e a entrada e saída de funcionários em um ambiente é de forma simples e intuitiva, durante o projeto foi de suma importância o planejamento prévio, para entender como o código se portaria de acordo com a necessidade, e as principais complicações estavam relacionados ao tempo, pois o período estipulado do projeto em si foram cinco dias, tornando inviável melhorias como um serviço de nuvem mais qualificado, pois a forma que o ThingSpeak suporta é por meio de fields, sendo disponibilizados até oito campos, enquanto se fosse utilizando um serviço de armazenamento como um banco de dados local, seria melhor configurado do que o serviço alocado no ThingSpeak, devido a possibilidade escolher campos optando por um sistema maior de identificação e suporte, porém apesar desse empecilho o projeto agregou de forma positiva na contribuição de material teórico e prático na área de indústria inteligente, já que mostrou eficiência de acordo com as limitações, além de cumprir o propósito de um sistema embarcado de baixo custo e de fácil implementação, trazendo consigo avanços significativos em pequenas e médias empresas que optam por métodos tecnológicos de gerenciamento funcionarios.

REFERÊNCIAS

GITHUB. BitDogLab-C. Disponível em:

<https://github.com/BitDogLab/BitDogLab-C/tree/main/display_oled/inc>

Acesso em: 16 de fev. de 2025.